SWDND501-NoSQL DATABASE DEVELOPMENT

LEARNING OUTCOME 1: PREPARE DATABASE ENVIRONMENT

Trainer: NGENDAHAYO Aloys

School: Msgr MUBILIGI CATHOLIC TSS

Academic year: 2024-2025

Lets start……

- **NoSQL** stands for "Not Only SQL." It refers to a variety of database technologies designed to store, retrieve, and manage data that is not structured in traditional relational databases.
  - Instead of using tables with rows and columns (as in relational databases), NoSQL databases can store data in various formats like key-value pairs, documents, wide- column stores, or graphs.
- **MongoDB** is a document database. It stores data in a type of JSON format called BSON.
- **or**

- **MongoDB** is a popular NoSQL document database that stores data in flexible, JSON-like documents. It allows for a flexible schema design, making it easy to accommodate changing requirements and store unstructured or semi-structured data.
  - **MongoDB** is known for its horizontal scalability, making it suitable for large-scale applications with growing data.
    - JSON stands for **J**ava**S**cript **O**bject **N**otation

Example of json format

- {
  "employees":[
  {"firstName":"John", "lastName":"Doe"},
  {"firstName":"Anna", "lastName":"Smith"},
  {"firstName":"Peter", "lastName":"Jones"}
  ]
  }

- **Availability**
  - In the context of databases, availability refers to the system's ability to remain operational and accessible, even during failures or high demand.
  - In NoSQL databases, availability is often prioritized over consistency in distributed systems
  - MongoDB uses replication and sharding to ensure high availability.
  - **Replication** involves copying data from one database server to multiple servers. This ensures that the system can continue to operate even if one server fails.

- In MongoDB, replication is achieved through **replica sets**, where one server (primary) accepts write operations, and multiple secondary servers maintain copies of the data.
- If the primary server fails, one of the secondaries can automatically take over as the new primary, ensuring no downtime.
- **Sharding** is a horizontal scaling method that divides large datasets across multiple servers, or **shards**.
- Each shard contains a portion of the data, allowing the system to handle massive amounts of data and traffic by distributing the load.
- **Sharding** is particularly useful when a single server can no longer store or process all the data due to resource limitations (e.g., CPU, memory, storage).

- **Example**: In MongoDB, a collection can be divided (sharded) across multiple servers based on a **shard key** (a field in the document).
- For instance, a large e-commerce dataset could be sharded across three servers based on a **customerID** , field, where each shard holds a subset of customers.

- [Shard1] customerID 1-100K
- [Shard2] customerID 100K-200K
- [Shard 3] customerID 200K-300K

- **Documents**
  - In MongoDB, a **document** is the basic unit of data, similar to a row in a relational database.
  - Documents are represented in a JSON-like format (BSON in MongoDB, a binary version of JSON).
- **What is BSON?**
- **BSON** stands for **Binary JSON**. It is a binary-encoded serialization format used to store and transport data. MongoDB, a NoSQL database, uses BSON to store documents on disk and transfer data over the network.
  - Each document can store key-value pairs, and the structure of each document can vary within the same collection.

- **Example:**
- **{**
- **"_id": "123",**
- **"name": "Alice",**
- **"age": 25,**
- **"address": {**
- **"street": "123 Main St",**
- **"city": "New York"**
- **}**
- **}**

- **Collection**
  - A **collection** is the equivalent of a table in a relational database system.
  - It is a group of documents. Unlike a table, there is no enforced schema, so documents in a collection can have different structures.
  - Collections organize documents into logical groups for querying and managing.
- **Example**: A collection called **users** might store documents representing individual user data.

- **Indexing**
  - Indexing in MongoDB helps to improve query performance.
  - It allows the database to quickly locate documents in a collection by creating special data structures (indexes) that store a small portion of the document's data in an efficient way.
  - Without indexes, MongoDB must scan every document in a collection, leading to slow query performance.
- **Example**: An index on the **name** field in the **users** collection will speed up queries searching for users by name.

- ✓ **Optimistic Locking**
  - Optimistic locking is a concurrency control mechanism where a system assumes that multiple transactions can complete without affecting each other.
  - In MongoDB, optimistic locking can be implemented using a field such as a version number **(v field)** that is incremented with every document update.
  - If two updates attempt to modify the same document at the same time, one will fail, ensuring that no conflicting changes are applied.
  - **Use Case**: Optimistic locking is useful in distributed systems where locking mechanisms should be avoided to maintain high performance and availability.

- **Relationships**
  - In NoSQL databases like MongoDB, relationships between data are not as strictly enforced as in relational databases. However, relationships can still be modeled.
  - There are two types of relationships:
    - **Embedding**: Storing related data in the same document. Best for one-to-many
    - **Referencing**: Storing related data in separate documents, using references (similar to foreign keys in relational databases).best for many-to-many.
  - **Example**: An order document may embed product details directly, or it may reference products stored in a separate collection.

- **Data Model**
  - The data model defines how data is structured, stored, and accessed in a database.
  - In MongoDB, the data model is flexible and can adapt to the application's needs. Unlike rigid schemas in SQL databases, MongoDB's data model allows for varied document structures within a collection.
  - **Example**: In MongoDB, one user document might have fields like **email**, **address** and **phone**, while another user document might have only an **email** and **phone**

- ✓ **Schema**
  - A schema defines the structure of data.
  - In traditional relational databases, schemas are strict, defining the fields and data types for each table.
  - In MongoDB, schema is flexible, meaning there is no enforced structure by default, but schema validation can be added if needed to impose constraints on the data.
  - **Example**: MongoDB allows documents in the same collection to have different fields, but you can enforce schema validation rules using JSON Schema.

- **Mongosh**

- Mongosh is the MongoDB shell, a command-line interface for interacting with MongoDB databases.
- It allows users to execute queries, manipulate data, and perform administrative tasks directly from the terminal.
- Mongosh provides a JavaScript-based environment to work with MongoDB.
- **Usage**: You can use Mongosh to connect to a MongoDB instance, insert or query documents, and perform database management tasks like creating indexes or viewing collection statistics.

✓ Identifying user requirements

1. Understand the Use Case

Determine the purpose of the database and the type of application it will support.

Examples:

- Real-time analytics for e-commerce.
- Storing IoT sensor data.
- Managing unstructured or semi-structured data like logs or user-generated content.

2. Analyze Data Characteristics

Identify the nature of the data to be stored:

- **Structured, Semi-Structured, or Unstructured**
- **Volume**: How much data will be stored (GBs, TBs, PBs)?
- **Velocity**: How fast is the data generated or ingested (e.g., real-time, batch)?
- **Variety**: Are there diverse data types, such as text, images, or JSON?

Example:

- Logs generated by a web server are semi-structured and high-velocity.
- User profile data for a social media app is structured but varies in size.

3. Define Functional Requirements

Identify what the database must do to meet user needs:

1. **Query Patterns**:
   - What types of queries will be run?
   - Example: Full-text search, range queries, geospatial queries.
2. **Data Relationships**:
   - Are there one-to-many or many-to-many relationships?
   - Example: Linking users to their orders or products to categories.
3. **CRUD Operations**:
   - Frequency of Create, Read, Update, Delete operations.
4. **Scalability**:
   - Is horizontal or vertical scaling needed?

4. Define Non-Functional Requirements

Include performance, reliability, and other system-level requirements:

1. **Performance**:
   - Query latency (e.g., <10ms for reads).
   - Write throughput (e.g., 10,000 writes per second).
2. **Availability**:
   - 99.99% uptime required for mission-critical applications.
3. **Consistency**:
   - Eventual consistency or strong consistency based on use case.
4. **Replication**:
   - Data redundancy for fault tolerance and high availability.
5. **Security**:
   - Role-based access control (RBAC), encryption, and auditing.

5. Map to NoSQL Data Models

Choose the most appropriate NoSQL data model based on the requirements:

1. **Document Model** (e.g., MongoDB):
   - Use for flexible schema and hierarchical data.
2. **Key-Value Model** (e.g., Redis):
   - Use for fast lookups with simple key-value pairs.
3. **Column-Family Model** (e.g., Cassandra):
   - Use for write-heavy applications and time-series data.
4. **Graph Model** (e.g., Neo4j):
   - Use for complex relationships and networks.

6. Determine Scaling and Performance Needs

- Will the database scale horizontally across servers?
- Is sharding required for massive datasets?
- Example:
  - For a global e-commerce platform, shard data by geographic region.
  - Use replication to maintain multiple copies for high availability.

7. Integration and Ecosystem

- Identify integrations with existing tools or frameworks:
  - Programming languages: Support for Java, Python, etc.
  - Analytics tools: Compatibility with BI tools or machine learning pipelines.
  - Cloud platforms: Does it need to integrate with AWS, GCP, or Azure?

8. Define Data Lifecycle Management

- Retention Policies: How long will data be kept?
  - Example: Logs older than 30 days are archived.
- Backup and Restore: Regular backups and disaster recovery plans.

9. Prototype and Validate

- Build a small prototype to test assumptions and validate requirements.
- Use performance benchmarks and test real-world query scenarios.

10. Document Requirements

- Clearly document functional and non-functional requirements for stakeholders.

Include:

- Use cases and examples.
- Data model diagrams.
- Query patterns.
- Scaling strategy.

1.Characteristics of Collections in MongoDB

- ✓ **Schema-Less Nature**:
  - Unlike traditional relational databases, MongoDB collections don't enforce a fixed schema. Each document can have different fields and structures.
- ✓ **Document Storage**:
  - Documents are stored inside collections. Each document represents a piece of data.
- ✓ **Explicit Creation**:
  - If a collection does not exist, MongoDB creates it automatically when you first store data in that collection.

- **Document Validation:**
  - By default, MongoDB collections do not enforce a strict schema.However, you can set up document validation rules during update and insert operations.
  - Schema validation ensures that documents meet specific criteria (e.g., required fields, data types) within a collection.
- **Unique Identifiers**:
  - Each collection is assigned an immutable UUID (Universally Unique Identifier).
  - The collection UUID remains the same across all members of a replica set and shards in a sharded cluster.

- ✓ **Sharding and Replication**:
  - Collections can be sharded (distributed across multiple servers) and replicated (duplicated across servers) to ensure high availability and scalability.
- ✓ **Schema Flexibility**

• Collections can store documents with different fields and structures.

• No predefined schema is required, allowing dynamic changes.

- ✓ **Dynamic Data Types**
  - Fields in documents within a collection can have varying data types.
  - Example: One document may store a field as a string, while another stores it as a number.
- ✓ **High Scalability**
- Collections are optimized for horizontal scaling using sharding.
- Data can be distributed across multiple servers.
- ✓ **Query Capabilities**
- Collections support flexible querying with operators for filtering, sorting, and projections.
- Includes full-text search, geospatial queries, and regex support.

- ✓ **Embedded and Referenced Data**
- Collections can store embedded documents (denormalized) or use references to link data (normalized).
- ✓ **Lifecycle Operations**
- Collections support CRUD (Create, Read, Update, Delete) operations efficiently.

2. Features of NoSQL Databases

- NoSQL databases have distinct features that make them suitable for certain use cases, particularly those involving large-scale, unstructured, or semi-structured data:
- ✓ Flexible Data Structures:
  - Unlike traditional relational databases, NoSQL databases don't enforce fixed tabular relationships.
  - Instead, they allow flexible data structures. This flexibility is essential for handling diverse data formats, whether structured, semi-structured, or unstructured.

- **Low Latency**:
  - NoSQL databases prioritize low-latency operations.
    They excel at quickly retrieving and storing data, making them suitable for real-time applications.
- **Horizontal Scalability**:
  - NoSQL databases are designed for scalability. As your data grows, you can easily distribute the workload across multiple servers or nodes.
  - This horizontal scaling ensures performance remains consistent even under heavy loads.

- **Large Number of Concurrent Users**:
  - NoSQL databases handle concurrent access efficiently. Whether it's thousands of users querying the database simultaneously or a high-throughput workload, NoSQL systems can cope effectively.
- **Optimized for Large Data Volumes**:
  - NoSQL databases accommodate large datasets, regardless of whether the data is structured (like JSON), semi-structured, or completely unstructured.
- **Distributed Architecture**:
  - NoSQL databases distribute data across nodes, allowing them to handle massive amounts of information. This architecture enhances fault tolerance and resilience.

- ✓ **Flexible Schema:**
- No strict schema requirements, allowing you to store data in various formats, including semi-structured and unstructured data.
- Easy adaptation to evolving data models as your application needs change.

- ✓ **High Performance:**
- Optimized for specific data models and workload patterns, offering fast read and write operations.
- Distributed caching and in-memory processing for low-latency data retrieval.

- ✓ **Rich Query Language:**
- Provides powerful query languages (often beyond SQL) for flexible data retrieval and manipulation.
- Enables complex queries and aggregations on large datasets.
- ✓ **Ease of Use:**
- Designed for developers with user-friendly APIs and tools for managing and interacting with the database.
- ✓ **Cost-Effectiveness:**
- Can be more cost-effective than traditional relational databases, especially for large-scale deployments.
- Pay-as-you-go pricing models and cloud-based offerings can help control costs.

✓ **Fault Tolerance:**

- Built-in fault tolerance and high availability through data replication and distribution.
- Minimizes downtime and ensures data resilience in case of failures.

✓ **Support for Big Data**

- Designed to handle massive volumes of structured, semi-structured, and unstructured data.
- Scalable to petabytes or more.

- ✓ **Varied Data Models**:
  - They support different data models, including key-value pairs, documents, wide-columns, and graphs, catering to various types of data and use cases.
- ✓ **Types of NoSQL Databases**
  - ❖ NoSQL databases are categorized based on their data models:

1. **Document-Based Databases**

   Stores data in JSON-like documents (flexible and hierarchical). Each document is self-contained, with fields and values that can differ across documents.

   - **Best for**:

     Applications requiring flexible and dynamic schemas, such as content management systems, e-commerce, and user profiles.

Examples:

- MongoDB
- Couchbase
- Amazon DocumentDB
- ArangoDB

2. Key-Value Databases

Stores data as simple key-value pairs. Each key is unique, and the value can be a string, JSON, or binary.

Optimized for fast lookups and writes.

Best for: Applications requiring low-latency operations like caching, session management, and real-time analytics.

Examples:

Redis ,DynamoDB ,Riak KV ,Berkeley DB

3. Column-Family Databases

Organizes data into rows and columns, similar to relational databases, but allows variable numbers of columns per row. Each row is identified by a unique key.

•**Best for**:

Use cases involving large-scale, write-heavy workloads such as IoT data, log processing, and time-series data.

- **Examples**:
  - Apache Cassandra
  - HBase
  - ScyllaDB

4. Graph Databases

Stores data as nodes (entities) and edges (relationships). Best suited for applications requiring complex relationship modeling.

- **Best for**:
  Social networks, recommendation engines, fraud detection, and network analysis.
- **Examples**:
  - Neo4j
  - Amazon Neptune
  - ArangoDB (multi-model with graph support)
  - OrientDB

5. Multi-Model Databases

Supports multiple types of data models (e.g., document, graph, key-value) in a single database system.

- **Best for**:
  Applications requiring diverse data representations without needing multiple database solutions.
- **Examples**:
  - ArangoDB
  - Couchbase
  - MarkLogic

6. Object-Oriented Databases

Stores data as objects, similar to object-oriented programming. Each object contains attributes (data) and methods (behavior).

- **Best for**:
  Applications with complex object hierarchies and those using object-oriented programming languages.
- **Examples**:
  - ObjectDB
  - Db4o

7. Time-Series Databases

Optimized for storing and analyzing time-stamped data, such as IoT sensor readings, financial data, or performance metrics.

- **Best for**:
  Time-series data analysis and monitoring applications.
- **Examples**:
  - InfluxDB
  - TimescaleDB
  - OpenTSDB

8. Search-Oriented Databases

Designed for full-text search and indexing of unstructured or semi-structured data.

- **Best for**:
  Search-heavy applications such as e-commerce product search or website indexing.
- **Examples**:
  - Elasticsearch
  - Solr

9. Event/Streaming Databases

Handles real-time data streams and events, often used for pub-sub systems and event-driven architectures.

- **Best for**:
  Real-time analytics and monitoring systems.
- **Examples**:
  - Apache Kafka
  - Amazon Kinesis

4. Data Types in NoSQL Databases

- Different NoSQL databases support various data types, but common data types include:
- ✓ **String**: Text data.
- ✓ **Integer/Float**: Numeric data for whole numbers and floating-point numbers.
- ✓ **Boolean**: True or false values.
- ✓ **Date/Time**: Timestamp or date values.
- ✓ **Binary Data**: Raw binary data (e.g., files, images).

- **Array**: A list of elements, which can be of any data type.
- **Object/Document**: Nested data structures, similar to JSON objects.
- **Set**: A collection of unique elements (e.g., Redis sets).
- **Geospatial Data**: Data representing geographic locations and shapes

common use cases for NoSQL databases:

1. Real-Time Applications

- **Description**: Applications requiring rapid data processing and retrieval with low latency.
- **Use Cases**:
  - Real-time analytics for e-commerce platforms (e.g., tracking user behavior).
  - Leaderboards for gaming platforms.
  - Stock trading platforms.
- **Suitable NoSQL Type**:
  - **Key-Value Stores** (e.g., Redis, DynamoDB) for caching and session storage.
  - **Document Databases** (e.g., MongoDB) for storing dynamic user data.

2. Content Management Systems (CMS)

- **Description**: Systems managing varied and dynamic content like blogs, articles, and media.
- **Use Cases**:
  - News portals with frequently updated articles.
  - E-learning platforms storing multimedia content.
- **Suitable NoSQL Type**:
  - **Document Databases** (e.g., MongoDB) for flexible schema and hierarchical data storage.

3. Internet of Things (IoT)

- **Description**: Applications ingesting large volumes of time-stamped sensor data.
- **Use Cases**:
  - Smart home devices logging temperature, humidity, etc.
  - Industrial IoT for equipment monitoring.
- **Suitable NoSQL Type**:
  - **Time-Series Databases** (e.g., InfluxDB, TimescaleDB).
  - **Column-Family Databases** (e.g., Cassandra) for handling high write throughput.

4. Social Media Platforms

- **Description**: Applications with complex relationships between users, posts, and interactions.
- **Use Cases**:
  - Friend/follower networks.
  - Storing and retrieving posts, likes, and comments.
- **Suitable NoSQL Type**:
  - **Graph Databases** (e.g., Neo4j) for relationships.
  - **Document Databases** (e.g., MongoDB) for storing user-generated content.

5. E-Commerce

- **Description**: Platforms requiring dynamic data storage, search capabilities, and user personalization.
- **Use Cases**:
  - Product catalogs with varied attributes.
  - User recommendations based on browsing history.
- **Suitable NoSQL Type**:
  - **Document Databases** (e.g., Couchbase) for product catalogs.
  - **Search-Oriented Databases** (e.g., Elasticsearch) for full-text search.

6. Gaming Applications

- **Description**: Applications requiring real-time leaderboards, player profiles, and multiplayer interactions.
- **Use Cases**:
  - Storing player stats, achievements, and rankings.
  - Real-time matchmaking for multiplayer games.
- **Suitable NoSQL Type**:
  - **Key-Value Stores** (e.g., Redis) for caching leaderboards.
  - **Document Databases** for storing player profiles.

7. Fraud Detection and Network Analysis

- **Description**: Applications that detect fraudulent transactions or analyze networks.
- **Use Cases**:
  - Analyzing transaction patterns in banking.
  - Monitoring communication networks for suspicious activity.
- **Suitable NoSQL Type**:
  - **Graph Databases** (e.g., Neo4j) for relationship-based queries.

8. Big Data and Analytics

- **Description**: Systems processing large-scale, write-heavy workloads for insights.
- **Use Cases**:
  - Log analysis for monitoring application performance.
  - Real-time analytics for marketing campaigns.
- **Suitable NoSQL Type**:
  - **Column-Family Databases** (e.g., Cassandra).
  - **Search-Oriented Databases** (e.g., Elasticsearch) for indexing and querying logs.

9. Healthcare and Genomics

- **Description**: Applications storing complex and diverse medical or genomic data.
- **Use Cases**:
  - Patient records with varying attributes.
  - Storing and querying genome sequences.
- **Suitable NoSQL Type**:
  - **Document Databases** (e.g., Couchbase) for flexible storage.
  - **Column-Family Databases** for large-scale data analysis.

10. Financial Applications

- **Description**: Applications requiring high availability and scalability for transactional data.
- **Use Cases**:
  - Storing user account data and transaction logs.
  - Real-time fraud detection.
- **Suitable NoSQL Type**:
  - **Key-Value Stores** (e.g., DynamoDB).
  - **Graph Databases** for transaction pattern analysis.

❖ Analysing NoSQL Database

Analyzing NoSQL Database Requirements

- The requirements analysis process is a crucial step in designing a NoSQL database that meets the needs of its users. Here's an overview of the process:
  - **Describe Requirements Analysis Process**
- The requirements analysis process involves a systematic approach to identifying, capturing, and documenting the requirements of a NoSQL database.
- The goal is to understand the needs of the stakeholders and end-users, and to define a clear set of requirements that can guide the design and development of the database.

Identify Key Stakeholders and End-Users.

- In the context of a NoSQL database project, stakeholders refer to individuals or groups who have a vested interest in the project's success or failure.
- They can be impacted by the project, or they can influence the project's outcome.
- examples of stakeholders who may be involved in a NoSQL database project:

1.Database Administrators (DBAs)
Responsible for managing, maintaining, and optimizing the NoSQL database environment.

2.Developers and Software Engineers
Use NoSQL databases to build scalable, high-performance applications.

3.Data Architects
Design the database structure, relationships, and schema to align with business needs.

4.IT Operations Teams
Ensure uptime, scalability, and fault tolerance for the NoSQL database systems.

5.Product Managers
Define database requirements to support product features and ensure customer satisfaction.

6. **Business Owners and Executives**
   Make decisions based on the database's ability to support business growth and cost efficiency.
7. **Data Scientists and Analysts**
   Use NoSQL databases for querying and analyzing large datasets to derive actionable insights.
8. **Compliance Officers**
   Ensure the database complies with regulatory and security standards (e.g., GDPR, HIPAA).
9. **Cloud Service Providers**
   Offer managed NoSQL solutions and support infrastructure scalability (e.g., AWS, Google Cloud).
10. **Third-Party Vendors and Consultants**
    Provide tools and expertise to integrate, monitor, and optimize NoSQL databases.

End-Users of NoSQL Databases

1. E-Commerce Customers
Benefit from personalized product recommendations and fast application performance.
2. Social Media Users
Experience real-time interactions and dynamic content delivery powered by NoSQL databases.
3. Gamers
Engage with real-time multiplayer games and leaderboards managed through NoSQL databases.
4. Healthcare Professionals
Access electronic health records and medical research data stored in NoSQL databases.
5. Financial Service Clients
Use secure and fast banking applications powered by distributed NoSQL databases.

6. Marketers and Advertisers

Analyze customer behavior to deliver targeted campaigns using data from NoSQL databases.

7. Logistics and Supply Chain Managers

Monitor and track inventory and shipments in real-time through NoSQL-powered systems.

8. IoT Device Users

Rely on NoSQL databases for processing data generated by smart devices.

9. Researchers and Academics

Manage and analyze large-scale datasets for scientific experiments and studies.

10. Streaming Platform Subscribers

Enjoy smooth, personalized video and audio streaming enabled by NoSQL databases.

- **2.Capture Requirements**
- Once the stakeholders and end-users have been identified, the next step is to capture their requirements. This can be done through a variety of techniques, including:
- ✓ **Interviews:** One-on-one interviews with stakeholders and end-users to gather information about their needs and expectations.
- ✓ **Surveys:** Online or paper-based surveys to gather information from a larger group of stakeholders and end-users.
- ✓ **Workshops:** Collaborative workshops to gather requirements and facilitate discussion among stakeholders and end-users.
- ✓ **Documentation review:** Review of existing documentation, such as business requirements documents, technical specifications, and user manuals.

- **3.Categorize Requirements**
- The captured requirements should be categorized into different types, such as:
- ✓ **Functional requirements:** Describe the features and functions of the NoSQL database, such as data storage, retrieval, and querying.
- ✓ **Non-functional requirements:** Describe the performance, security, and scalability requirements of the NoSQL database.
- ✓ **User interface requirements:** Describe the user interface and user experience requirements of the NoSQL database.

- **4.Interpret and Record Requirements**
- The captured and categorized requirements should be interpreted and recorded in a clear and concise manner. This can be done using a variety of tools and techniques, such as:
- ✓ **Requirements management tools:** Tools like JIRA, Trello, or Asana to manage and track requirements.
- ✓ **Use cases:** Written descriptions of how the NoSQL database will be used by end-users.
- ✓ **User stories:** Brief descriptions of the features and functions of the NoSQL database from the end-user's perspective.

- **5.Validate Requirements**
- The final step is to validate the requirements to ensure that they are complete, accurate. This can be done through a variety of techniques, including:
- ✓ **Reviews:** Peer reviews of the requirements documentation to ensure that it is complete and accurate.
- ✓ **Prototyping:** Building a prototype of the NoSQL database to test and validate the requirements.
- ✓ **Testing:** Testing the NoSQL database against the requirements to ensure that it meets the needs of the stakeholders and end-users.

❑ Performing Data Analysis and Implementing Data Validation for a NoSQL Database

❖ Performing Data Analysis

- Data analysis is a crucial step in designing a NoSQL database that meets the needs of its users.
- The goal of data analysis is to understand the structure, relationships, and patterns in the data, and to identify the requirements for storing and processing that data.
- Here are some steps involved in performing data analysis for a NoSQL database:

1. Data Profiling

- ✓ **Data type analysis:** Identify the data types of each attribute, such as strings, integers, dates, etc.

- **Data distribution analysis:** Analyze the distribution of data values, such as the frequency of each value, the range of values, etc.
- **Data relationship analysis:** Identify the relationships between different attributes, such as one-to-one, one-to- many, many-to-many, etc.

2. Data Quality Analysis

- **Data cleanliness analysis:** Identify any errors, inconsistencies, or missing values in the data.
- **Data redundancy analysis:** Identify any duplicate or redundant data.
- **Data integrity analysis:** Identify any data that is inconsistent or contradictory.

- **Data Modeling**
- ✓ **Entity-relationship modeling:** Identify the entities, attributes, and relationships in the data.
- ✓ **Document-oriented modeling:** Identify the documents, fields, and relationships in the data.
- ✓ **Graph modeling:** Identify the nodes, edges, and relationships in the data.
- ➢ Graph modeling is a technique used to represent complex relationships between data entities in a NoSQL database.
- ➢ In a graph, nodes represent individual data entities, such as people, places, things, or concepts.

- Edges represent the relationships between nodes in the graph.
- Edges can be directed (i.e., they have a direction from one node to another) or undirected (i.e., they do not have a direction).

- **Implementing Data Validation**
- Data validation is an essential step in ensuring the quality and integrity of the data in a NoSQL database.
- The goal of data validation is to ensure that the data is accurate, complete, and consistent.
  - Here are some steps involved in implementing data validation for a NoSQL database:

1. Data Validation Rules

- **Data type validation:** Validate the data type of each attribute, such as strings, integers, dates, etc.
- **Data range validation:** Validate the range of values for each attribute, such as minimum and maximum values.
- **Data format validation:** Validate the format of each attribute, such as date formats, phone number formats, etc.

2. Data Validation Techniques

- **Client-side validation:** Validate data on the client-side before sending it to the server.
- **Server-side validation:** Validate data on the server-side before storing it in the database.
- **Database-level validation:** Validate data at the database level using constraints, triggers, and stored procedures.

3. Data Quality Checks

- ✓ **Data normalization:** Normalize the data to ensure consistency and reduce redundancy.
- ✓ **Data standardization:** Standardize the data to ensure consistency and accuracy.
- ✓ **Data cleansing:** Cleanse the data to remove errors, inconsistencies, and missing values.

- By performing data analysis and implementing data validation, you can ensure that the data in your NoSQL database is accurate, complete, and consistent, and that it meets the needs of its users.

- **Preparing Database Environment**
- **Identifying MongoDB Scalability**
- MongoDB is a scalable NoSQL database that can handle large amounts of data and high traffic.
- Here are some factors that contribute to MongoDB's scalability:

- **1.Horizontal Scaling**
- ✓ **Sharding:** MongoDB allows you to split data across multiple servers, known as shards, to distribute the load and improve performance.
- ✓ **Replication:** MongoDB supports replication, which allows you to create multiple copies of data across different servers to ensure high availability and fault tolerance.
- **2.Vertical Scaling**
- ✓ **Increasing resources:** MongoDB can take advantage of increased resources, such as CPU, memory, and storage, to improve performance.
- ✓ **Optimization:** MongoDB provides various optimization techniques, such as indexing and caching, to improve query performance.

- **Scalability Features**

- ✓ **Auto-sharding:** MongoDB can automatically split data across multiple shards as the dataset grows.
- ✓ **Load balancing:** MongoDB supports load balancing, which distributes incoming traffic across multiple servers to improve performance and availability.
- ✓ **GridFS:** MongoDB's GridFS allows you to store large files and scale horizontally to handle high traffic.

- **Scalability Considerations**
- ✓ **Data growth:** Plan for data growth and ensure that the database can handle increasing amounts of data.
- ✓ **Traffic patterns:** Analyze traffic patterns and ensure that the database can handle peak traffic and sudden spikes.
- ✓ **Resource utilization:** Monitor resource utilization and ensure that the database is optimized for performance and efficiency.

Setting up MongoDB Environment: Shell, Compass, and Atlas

- Setting up a MongoDB environment involves installing and configuring the necessary tools and services to interact with a MongoDB database. You can set up MongoDB in three main environments, depending on your needs: MongoDB Shell, MongoDB Compass, and MongoDB Atlas.
  - **MongoDB Shell:** The MongoDB Shell, also known as the "**mongosh**", is a command-line interface for interacting with a MongoDB database.

Steps to Set Up MongoDB Shell:

1. **Install MongoDB**: First, download and install MongoDB from the MongoDB website.
   - Make sure mongod (MongoDB daemon) is running to start your MongoDB server.
   - Use mongosh to interact with your MongoDB instance via the shell.
2. **Start MongoDB**:
   - Launch the MongoDB service with: mongod.
   - Connect to the MongoDB server using: mongosh.
3. **Basic Commands**:
   - **show dbs** to list all databases.
   - **use <database_name>:** to switch to or create a new database.
   - **db.createCollection('myCollection')** to create a collection.
   - **db.myCollection.insert({name: "example"})** to insert a document into a collection.
   - **show collections:**Lists all collections in the current database.
   - **db.collection.find():**Retrieves data from a collection.

- **Compass Environment**
  - **MongoDB Compass:** MongoDB Compass is a graphical user interface (GUI) for MongoDB that provides a visual representation of your database and allows you to interact with it.

Steps to Set Up MongoDB Compass:

1. **Download MongoDB Compass**: You can download Compass from the MongoDB website.
2. **Connect to Your Database**:
   - Use the connection string mongodb://localhost:27017 for a local instance, or use the string provided by your cloud or remote MongoDB.
3. **Explore and Manage**:
   - Use Compass to create and manage databases, run queries, and visually analyze your data. It offers an intuitive interface for data inspection and query building.
   - Navigate collections, indexes, and documents with ease using its GUI.

- **Atlas Environment**
  - **MongoDB Atlas:** MongoDB Atlas is a cloud-based MongoDB service that provides a fully managed database- as-a-service (DBaaS) solution.

Steps to Set Up MongoDB Atlas:

1. **Sign Up for MongoDB Atlas**:
   - Go to MongoDB Atlas and sign up for an account.
2. **Create a Cluster**:
   - Once signed in, create a new cluster by selecting your cloud provider (AWS, Azure, GCP) and region.
   - Atlas will automatically manage, scale, and back up your cluster.
3. **Connect to Your Cluster**:
   - After setting up a cluster, connect using the provided connection string, which you can use in mongosh, Compass, or your application.
   - Example connection string: mongodb+srv://username:password@cluster0.mongodb.net/test.
4. **Manage via Atlas Dashboard**:
   - Use the Atlas dashboard to monitor performance, set alerts, and scale your cluster as needed. You can easily adjust your cluster size with a few clicks.

LEARNING OUTCOME 2: DESIGN NOSQL DATABASE

2.1 Selecting tools of drawing databases.

- When selecting tools for drawing database models in NoSQL environments, it is important to pick tools that support NoSQL's schema-less structure and facilitate the creation of non-relational models, such as document-based, key-value, wide-column, and graph databases.

2.1.1 Identify NoSQL drawing tools

- Here are some popular NoSQL database modeling and diagramming tools that can help you visualize the structure and relationships within NoSQL databases:

1. Lucidchart
2. Draw.io (with NoSQL templates)
3. Studio 3T (for MongoDB)
4. Moon Modeler
5. DbSchema
6. EdrawMax
7. Luna Modeler
8. Moon Modeler
9. Visual Paradigm
10. StarUML
11. Microsoft Visio

2.1.2 Installation of Edraw Max drawing tool

- **Edraw Max** is an all-in-one diagramming tool, offering support for multiple types of diagrams, including flowcharts, mind maps, and database diagrams. While not specifically tailored for NoSQL databases, it can be used to visualize NoSQL data models.

- **Steps to Install Edraw Max:**

1. Download:

- Go to the Edraw Max official website and download the installer for your operating system (Windows, macOS, or Linux).

2. Install:

- Run the installer after the download is complete.
- Follow the installation wizard:
  - Choose the installation directory.
  - Accept the license agreement.
  - Select any additional features or components you wish to install.

3. Launch:

- Once installed, open Edraw Max.
- You can choose the type of diagram you wish to create from the home screen.
- If you are creating database models, search for database diagrams in the templates or create your own custom diagram.

2.2 Creating Conceptual Data Model.

- **Data modeling in MongoDB** is the process of designing and creating the structure of collections and documents that will be stored in the database.
- There is no need to build a schema before adding data to **MongoDB database** because MongoDB is flexible. This means that MongoDB supports a **dynamic database schema**.

❖ MongoDB Data Model Designs

- For modeling data in MongoDB, two strategies are available. These strategies are different and it is recommended to analyze scenario for a better flow.
- The two methods for data model design in MongoDB are:
  - Embedded Data Model
  - Normalized Data Model

1. Embedded Data Model

This method, also known as the **de-normalized** data model, allows you embedd all of the **related documents in a single document**.

Embedded Data Model

- **Structure:** Related data is stored within a single document, often nested within other documents.
- **Benefits:**
  - **Improved Read Performance:** Reduces the number of database round trips required to fetch related data, leading to faster query execution.
  - **Atomic Operations:** Updates to related data can be performed atomically within a single document, ensuring data consistency.
  - **Simplicity:** Easier to implement for simple relationships.
- **Drawbacks:**

- **Data Duplication:** Can lead to data redundancy, which might increase storage costs and complexity in maintaining data consistency.
- **Scalability Limitations:** As data grows, large documents can become less efficient to query and update.
- **Complexity for Complex Relationships:** Handling complex relationships can become challenging with embedded documents.

- **Embedded Data Model example 1**

If we obtain student information in three different documents, Personal details, Contact, and Address, we can embed all three in a single one, as shown below.

```
{
    _id: ,
    Std_ID: "STD001"
    Personal_details:{
        First_Name: "RWAKA",
        Last_Name: "Angelo",
        Date_Of_Birth: "1999-08-26"
    },
    Contact: {
        e-mail: "rwakaang123@gmail.com",
        phone: "9987645673"
    },
    Address: {
        city: "Kigali",
        Area: "Nyarugenge",
        State: "Gitega"
    }
}
```

- **Example 2:**

```
{
  "postId": 1,
  "title": "MongoDB Data Models",
  "comments": [
    {"userId": 101, "text": "Great post!"},
    {"userId": 102, "text": "Very informative."}
  ]
}
```

- **example 3**

- Embedded documents store related data in a single document structure. A document can contain arrays and sub-documents with related data.
  These **denormalized** data models allow applications to retrieve related data in a single database operation.

- **Normalized Data Model (References)**

- In a normalized data model, object references are used to express the **relationships between documents and data objects**. Because this approach **reduces data duplication**, it is relatively simple to document **many-to-many relationships** without having to repeat content.

- **Normalized Data Model**

- **Structure:** Data is divided into multiple collections, with relationships between collections defined by references (usually object IDs).

- **Benefits:**
  - **Data Integrity:** Reduces data redundancy and ensures data consistency.
  - **Scalability:** Can handle large datasets more efficiently by breaking down data into smaller, more manageable chunks.
  - **Flexibility:** Easier to adapt to changing data requirements.
- **Drawbacks:**
  - **Increased Query Complexity:** Requires multiple database round trips to fetch related data, potentially impacting performance.
  - **Increased Complexity in Data Modeling:** More complex to design and implement, especially for complex relationships.
- **Normalized (References) Data Model Example 1**

Here we have created multiple collections for storing students data which are linked with _id.

Student:

```
{
    _id: <StudentId101>,
    Std_ID: "STD001"
}
```

Personal_Details:

```
{
    _id: <StudentId102>,
    stdDocID: " StudentId101",
    First_Name: "RWAKA",
    Last_Name: "Angelo",
    Date_Of_Birth: "1999-08-26"
}
```

Contact:

```
{
    _id: <StudentId103>,
    stdDocID: " StudentId101",
    e-mail: " rwakaang123@gmail.com",
    phone: "9987645673"
}
```

Address:

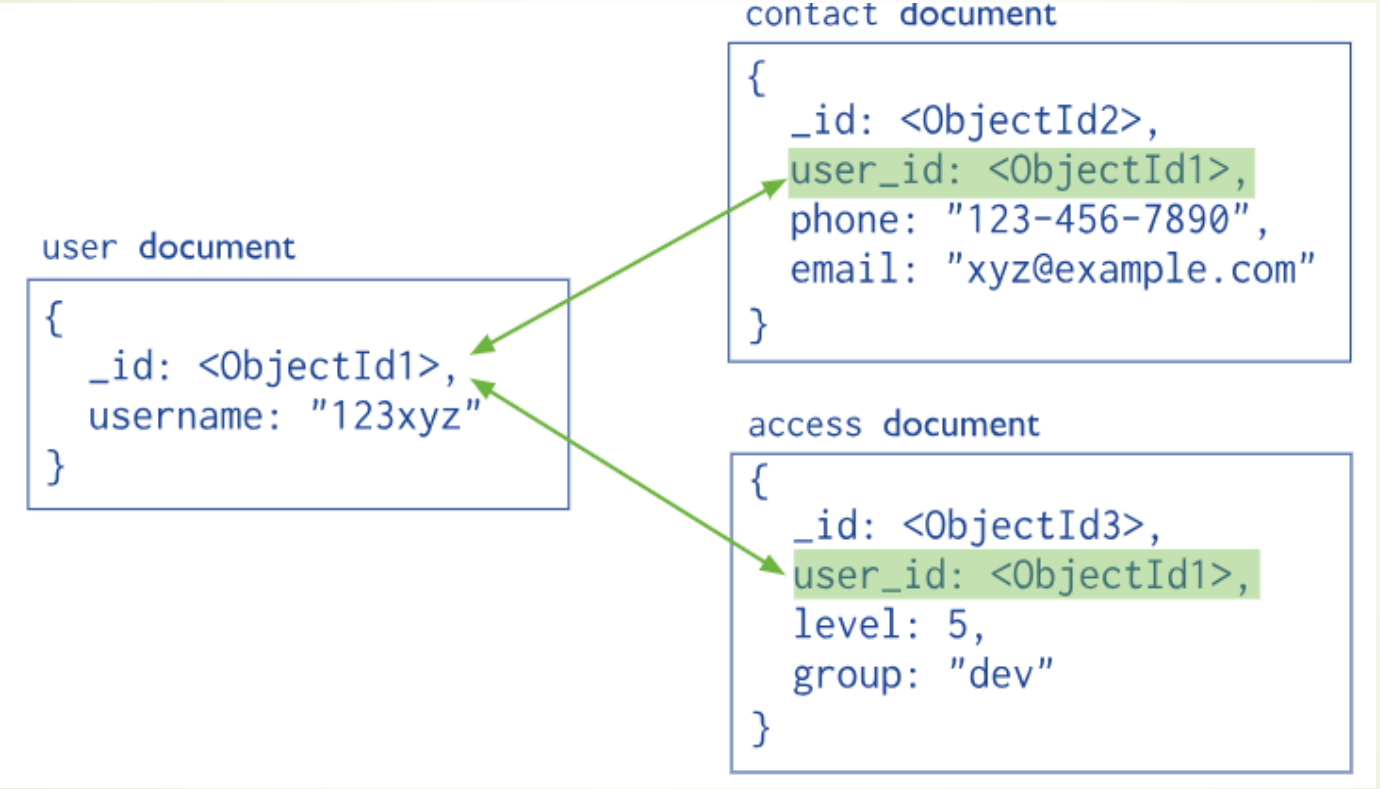
```
{
    _id: <StudentId104>,
    stdDocID: " StudentId101",
    city: "Kigali",
    Area: "Nyarugenge",
    State: "Gitega"
}
```

Example 2:

```
// Post document
{
  "postId": 1,
  "title": "MongoDB Data Models"
}
// Separate comment documents
{
  "commentId": 201,
  "postId": 1,
  "userId": 101,
  "text": "Great post!"
```

```
}
{
  "commentId": 202,
  "postId": 1,
  "userId": 102,
  "text": "Very informative."
}
```

- **Normalized (References) Data Model Example 3**



- **Steps for creating Conceptual Data Model in MongoDB.**

A conceptual data model (CDM) is a visual representation of an organization's data requirements and the relationships between its business entities.

While **MongoDB is a NoSQL database** that doesn't strictly adhere to the traditional relational data model, we can still apply similar principles to design a conceptual data model that aligns with its document-oriented nature. Here's a step-by-step guide to help you achieve this:

1. Identify Collections

In MongoDB, a **collection** is analogous to a table in relational databases but designed to store documents (JSON-like objects). Each collection should represent a major entity or group of entities in your system.

Example: Collections for an e-commerce platform:

- Users
- Products
- Orders
- Payments
- Categories

2. Model Entity Relationships

MongoDB does not enforce relationships like foreign keys in SQL, but you can still represent relationships between entities by:

- **Embedding Documents**: When entities have a **one-to-few** or **tightly coupled** relationship, embed related documents inside a parent document.
- **Referencing**: When relationships are **one-to-many** or **many-to-many** (like products and categories), store references (e.g., product_id) to related documents instead of embedding.

Example:

- A User may embed the Address directly in the document.
- An Order may reference a Product by storing the product's product_id.

Modeling Relationships Example:

```
{
        "user_id": "U123",
        "name": "John Doe",
        "email": "john@example.com",
        "orders": [
          {
            "order_id": "O1001",
            "date": "2023-01-15",
            "products": [
              { "product_id": "P5001", "name": "Laptop", "price": 1000 },
              { "product_id": "P5002", "name": "Mouse", "price": 25 }
            ]
          }
        ]
}
```

3. Define Sharding and Replication

- MongoDB supports **horizontal scaling** via sharding and **replication** for high availability.
- **Sharding**: Split large collections across multiple servers. Typically, you'll shard on:
  - Fields with a large number of unique values (e.g., user_id, order_id).
  - Consider sharding large datasets like Orders based on order_id or user_id for a user-centric view.
- **Replication**: Create multiple copies of your data across different nodes to improve data redundancy and read scalability.
  - For collections where read performance is critical (e.g., Products), set up replication to improve query efficiency and fault tolerance.

4. **Visualize High-Level Data Model**

1. UML Class Diagrams

- A **UML Class Diagram** helps to visualize the system's objects (or collections in the case of NoSQL) and their relationships. While traditional relational models focus on tables and columns, NoSQL models often involve collections (for document-based NoSQL like MongoDB) or other structures like key-value pairs or wide-column models.

➨ **Key Elements of a UML Class Diagram:**

- **Classes**: Represent entities (collections in NoSQL).
- **Attributes**: Represent fields or properties within the class (document fields or key-value pairs).
- **Associations**: Represent relationships between entities (1-to-1, 1-to-many, or many-to-many).

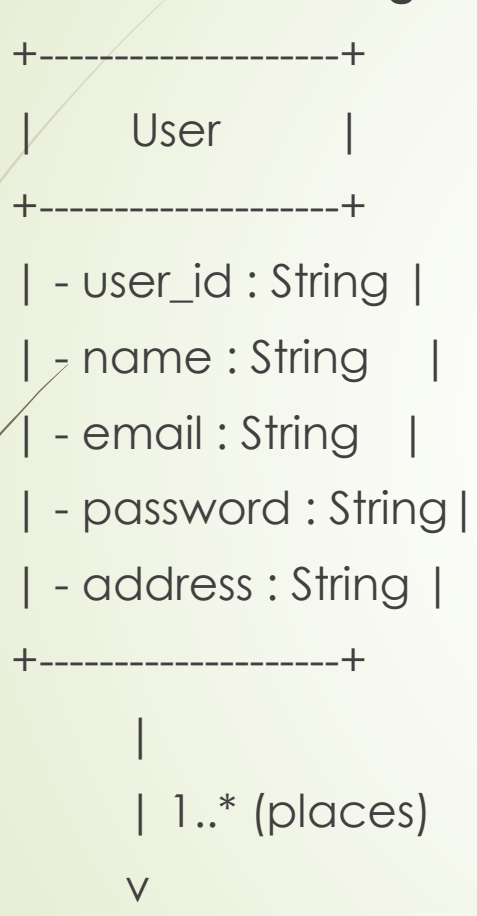
- **Example: E-commerce System**

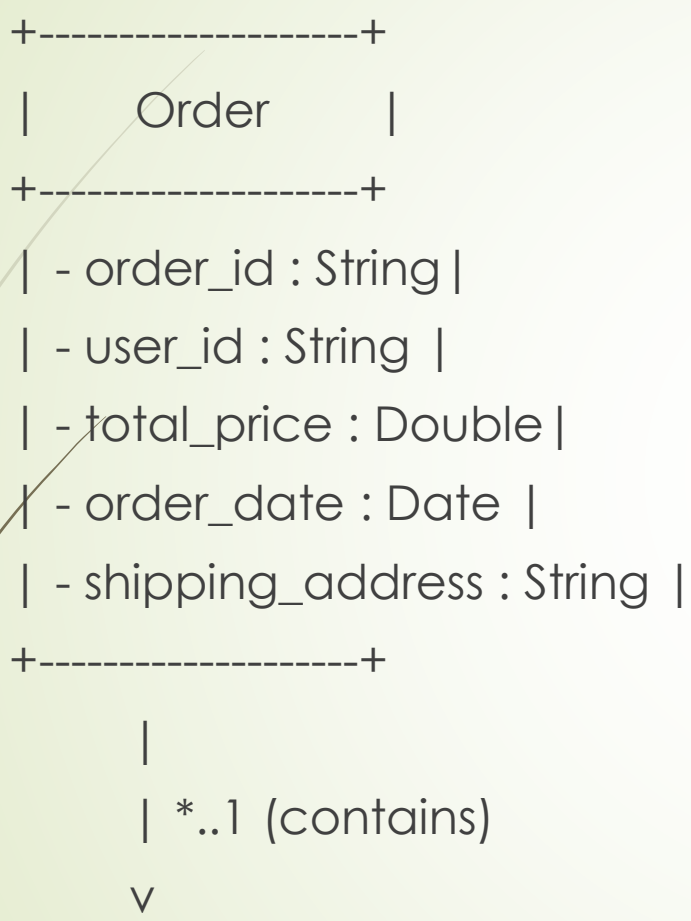
Consider an **E-commerce** application with the following entities:

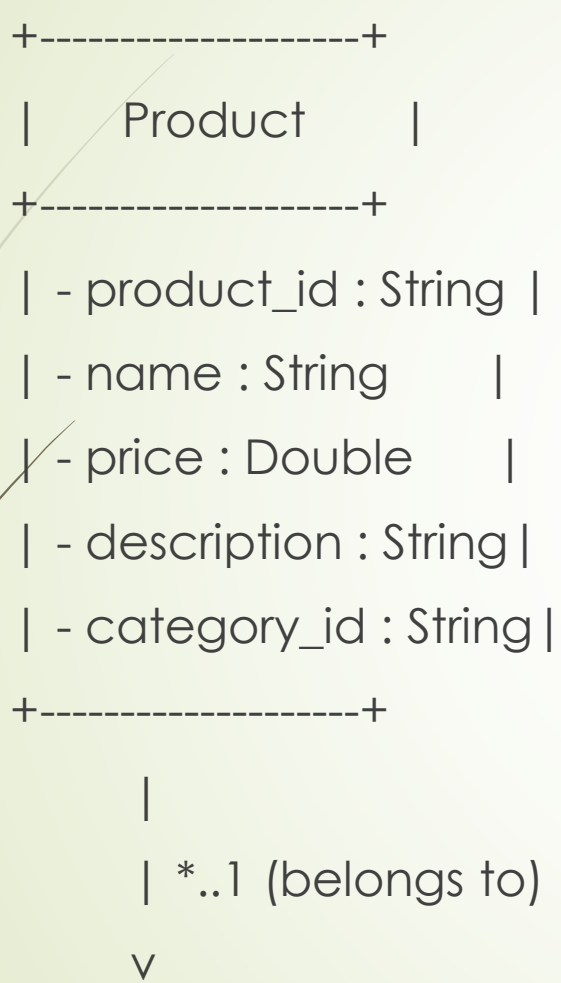
- **User**
- **Product**
- **Order**
- **Category**

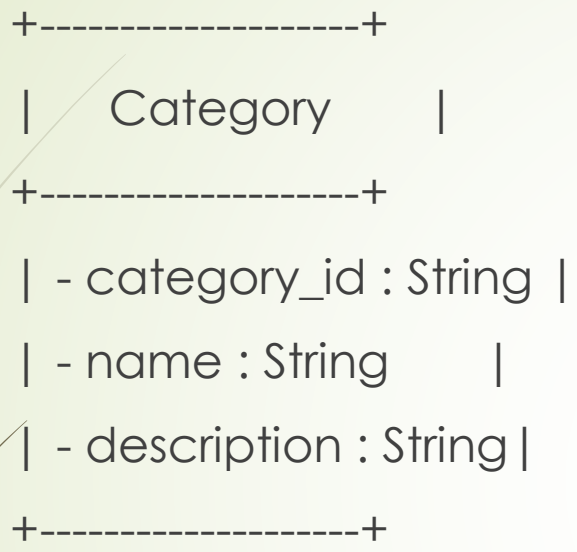
Each of these entities will have associated attributes, and we'll model the relationships between them.

- **UML Class Diagram Example:**




Order
- order_id : String
- user_id : String
- total_price : Double
- order_date : Date
- shipping_address : String
*..1 (contains)


+------------------+
| Product |
+------------------+
| - product_id : String |
| - name : String |
| - price : Double |
| - description : String |
| - category_id : String |
+------------------+
|
| *..1 (belongs to)
v


Category
- category_id : String
- name : String
- description : String

- **Explanation:**

- **User** → **Order**: A user can place many orders, so there's a **1-to-many** relationship between **User** and **Order** (indicated by 1..*).
- **Order** → **Product**: An order contains many products, which is a **many-to-many** relationship. This could be implemented in a NoSQL database by storing an array of product IDs in the order document (denormalization).
- **Product** → **Category**: A product belongs to a single category, so there is a **many-to-one** relationship between **Product** and **Category** (indicated by *..1).

2. Data Flow Diagrams (DFDs)

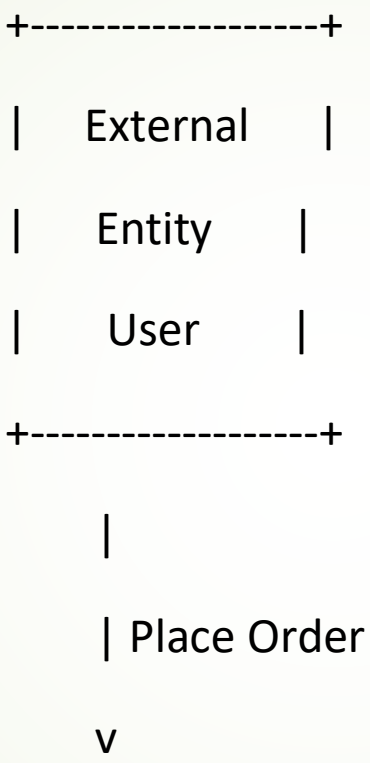
- A **Data Flow Diagram (DFD)** shows how data moves through the system. It describes the **input**, **processes**, and **output** that the system uses to transform data from one state to another. DFDs are helpful in understanding the flow of data and ensuring that every piece of data is processed correctly.

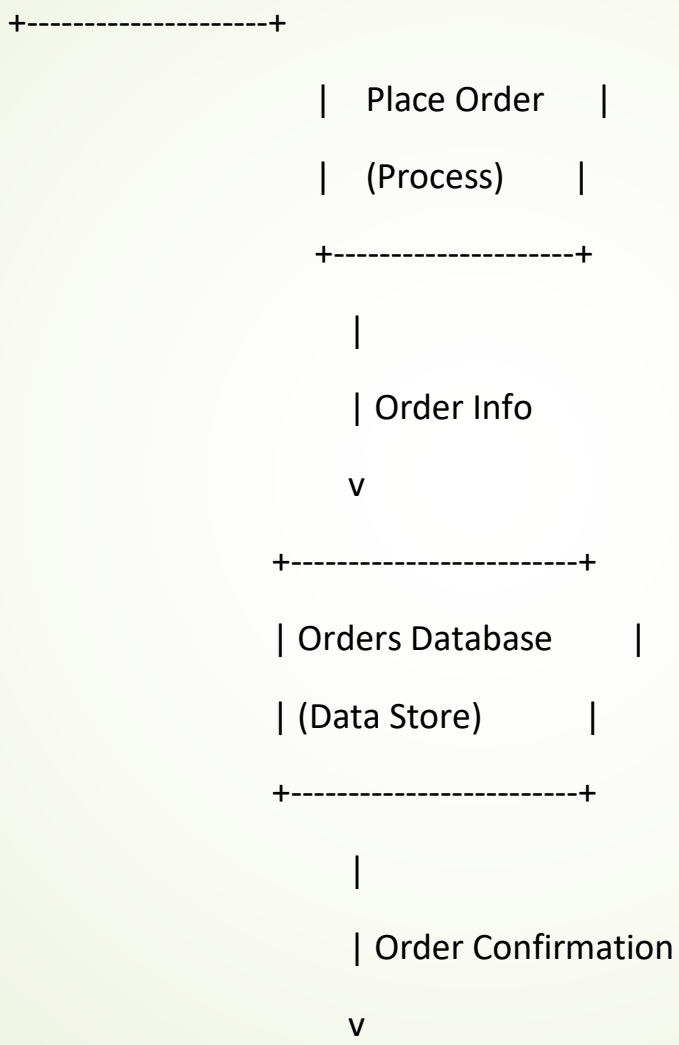
- **Key Elements of a DFD:**
  - **External Entities**: Sources or destinations of data (e.g., Users, External systems).
  - **Processes**: Actions or transformations that the system performs (e.g., Create Order, Process Payment).
  - **Data Stores**: Where data is stored (e.g., Products, Users, Orders).
  - **Data Flows**: Arrows representing the movement of data between entities, processes, and stores.

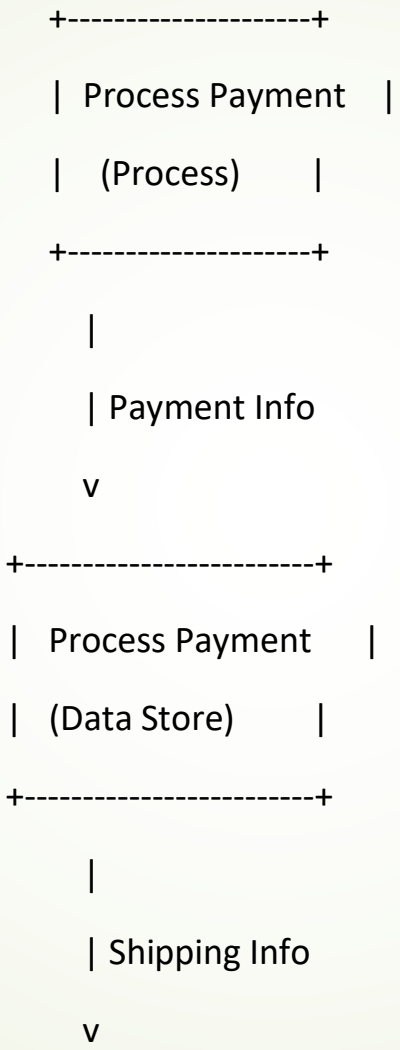
- **Example: E-commerce System**

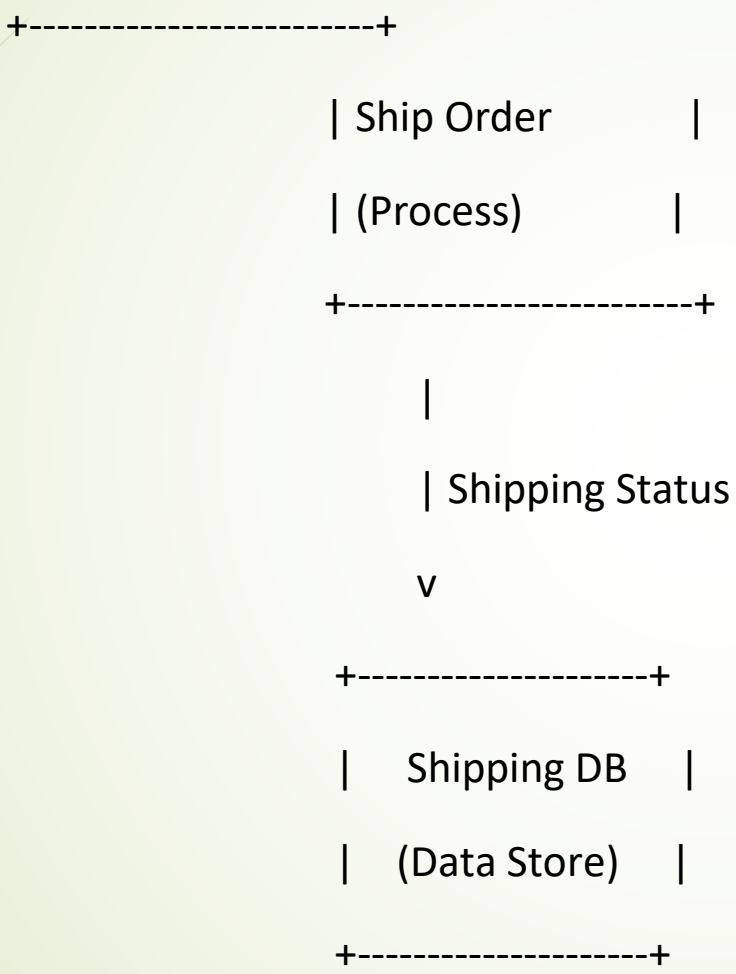
In this system:

- **External Entity**: User (customer who places orders)
- **Processes**:
  - Search Products
  - Place Order
  - Process Payment
  - Ship Order
- **Data Stores**:
  - Products DB
  - Users DB
  - Orders DB


External
Entity
User
Place Order


Place Order
(Process)
Order Info
Orders Database
(Data Store)
Order Confirmation


Process Payment
(Process)
Payment Info
Process Payment
(Data Store)
Shipping Info


Ship Order
(Process)
Shipping Status
Shipping DB
(Data Store)

- **Explanation:**

- **User → Place Order**: The external entity **User** interacts with the system to place an order.
- **Place Order → Orders Database**: The **Place Order** process stores the order details in the **Orders Database**.
- **Process Payment**: After the order is placed, the **Process Payment** handles payment transactions and stores payment information in a **Payment Data Store**.
- **Ship Order**: After the payment is processed, the system proceeds to **Ship Order**, storing the shipping information in a **Shipping Data Store**.

- **References:**

- *MongoDB, Inc. (n.d.). Data modeling introduction. MongoDB Documentation. Retrieved January 8, 2025, from https://www.mongodb.com/docs/manual/core/data-modeling-introduction/*
- *The Apache Software Foundation. (n.d.). Cassandra data modeling. Retrieved January 8, 2025, from https://cassandra.apache.org/_/index.html*
- *Visual Paradigm. (n.d.). What is class diagram?. Retrieved January 8, 2025, from https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-class-diagram/*
- *Lucidchart. (n.d.). Data flow diagrams (DFD). Retrieved January 8, 2025, from https://www.lucidchart.com/pages/dfd*
- *DataStax Academy. (n.d.). Data modeling in NoSQL. Retrieved January 8, 2025, from https://academy.datastax.com/*